CI/CD con Jenkins

Pruebas con JaCoCo, validación con SonarQube, publicación en Nexus y despliegue con Docker

Enlace repositorio

https://github.com/Johan-Guzman/taller-jenkins-sonar-deploy-A00401480-A00400842

Integrantes

Johan Stiven Guzmán — A00401480
Adri Jhoanny Martinez Murillo — A00400842

Docente

Juan José Valencia

Asignatura

Ingesoft V

Cali, Valle Del Cauca - Colombia
21 de octubre de 2026

Punto 2.1 – Fase 1: Preparación del entorno y SonarQube Server en Docker

En esta fase dejamos listo todo lo que el pipeline necesita antes de escribir una sola etapa del `Jenkinsfile`: el servidor SonarQube corriendo en Docker, el proyecto `product-api` registrado, un Quality Gate estricto y la comunicación entre SonarQube y Jenkins mediante un webhook.

Verificación de Docker y despliegue de SonarQube

Primero comprobamos que Docker respondiera correctamente con `docker ps`. La lista de contenedores aparece vacía, lo que confirma que el servicio funciona y que aún no había nada en ejecución.

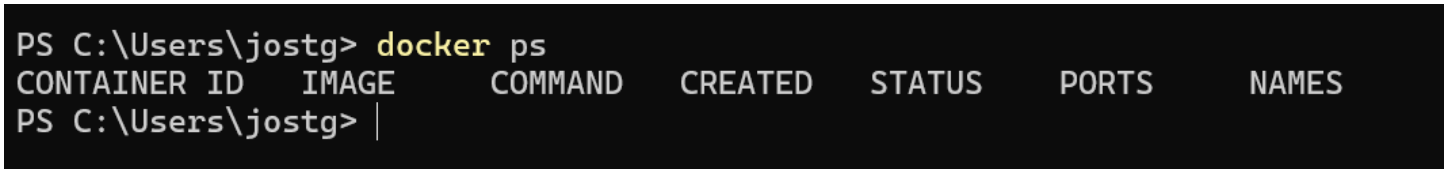


Figura 1. *Verificación de Docker con docker ps: la lista de contenedores aparece vacía antes del despliegue.*

Luego levantamos SonarQube Community Edition con el archivo `docker-compose.yml` del repositorio, usando `docker compose up -d sonarqube`. Docker descargó la imagen `sonarqube:10-community`, creó los volúmenes persistentes de datos, extensiones y logs, creó la red y arrancó el contenedor. La advertencia sobre el atributo `version` del compose es informativa y no afectó el despliegue.

```
docker ps
docker compose up -d sonarqube
docker logs -f sonarqube
```

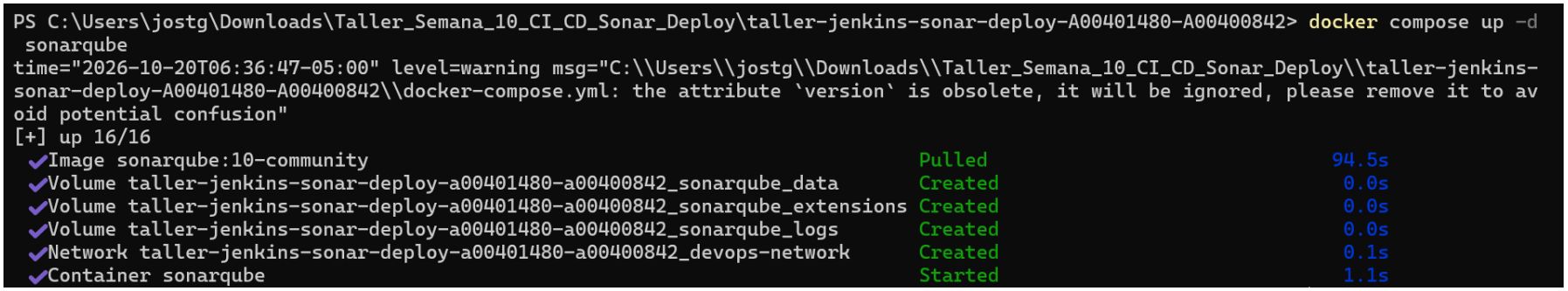


Figura 2. *Despliegue de SonarQube con docker compose up -d sonarqube: imagen descargada, volúmenes y red creados y contenedor iniciado.*

Monitoreamos el arranque con `docker logs -f sonarqube` hasta ver el mensaje `SonarQube is operational`, que indica que el servidor web y el Compute Engine ya estaban activos. En el mismo log aparece la advertencia de que todavía se usaban las credenciales de administrador por defecto, que corregimos en el siguiente paso.

```
2026.10.20 11:39:44 INFO  app[][o.s.a.ProcessLauncherImpl] Launch process[COMPUTE_ENGINE] from [/opt/sonarqube]: /opt/java/openjdk/b
n/java -Djava.awt.headless=true -Dfile.encoding=UTF-8 -Djava.io.tmpdir=/opt/sonarqube/temp -XX:-OmitStackTraceInFastThrow --add-open
=java.base/java.util=ALL-UNNAMED --add-exports=java.base/jdk.internal.ref=ALL-UNNAMED --add-opens=java.base/java.lang=ALL-UNNAMED --
dd-opens=java.base/java.nio=ALL-UNNAMED --add-opens=java.base/sun.nio.ch=ALL-UNNAMED --add-opens=java.management/sun.management=ALL-
NNAMED --add-opens=jdk.management/com.sun.management.internal=ALL-UNNAMED -Xmx512m -Xms128m -XX:+HeapDumpOnOutOfMemoryError -Dhttp.n
nProxyHosts=localhost|127.*|[::1] -cp ./lib/sonar-application-10.7.0.96327.jar:/opt/sonarqube/lib/jdbc/h2/h2-2.2.224.jar org.sonar.c
.app.CeServer /opt/sonarqube/temp/sq-process4807826118550396993properties
2026.10.20 11:39:45 WARN  app[][startup] ##########################################################################################
########################
2026.10.20 11:39:45 WARN  app[][startup] Default Administrator credentials are still being used. Make sure to change the password or
deactivate the account.
2026.10.20 11:39:45 WARN  app[][startup] ##########################################################################################
########################
2026.10.20 11:39:45 INFO  web[][o.s.s.p.Platform] Web Server is operational
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by org.sonar.process.PluginSecurityManager (file:/opt/sonarqube/lib/sonar-applic
tion-10.7.0.96327.jar)
WARNING: Please consider reporting this to the maintainers of org.sonar.process.PluginSecurityManager
WARNING: System::setSecurityManager will be removed in a future release
2026.10.20 11:39:45 INFO  ce[][o.s.p.ProcessEntryPoint] Starting Compute Engine
2026.10.20 11:39:45 INFO  ce[][o.s.ce.app.CeServer] Compute Engine starting up...
2026.10.20 11:39:46 INFO  ce[][o.s.d.DefaultDatabase] Create JDBC data source for jdbc:h2:tcp://127.0.0.1:9092/sonar;NON_KEYWORDS=VA
UE
2026.10.20 11:39:46 INFO  ce[][c.z.h.HikariDataSource] HikariPool-1 - Starting...
2026.10.20 11:39:46 INFO  ce[][c.z.h.p.HikariPool] HikariPool-1 - Added connection conn0: url=jdbc:h2:tcp://127.0.0.1:9092/sonar use
=
2026.10.20 11:39:46 INFO  ce[][c.z.h.HikariDataSource] HikariPool-1 - Start completed.
2026.10.20 11:39:46 WARN  ce[][o.s.db.dialect.H2] H2 database should be used for evaluation purpose only.
2026.10.20 11:39:47 INFO  ce[][o.s.s.p.ServerFileSystemImpl] SonarQube home: /opt/sonarqube
2026.10.20 11:39:47 INFO  ce[][o.s.c.c.CePluginRepository] Load plugins
2026.10.20 11:39:49 INFO  ce[][o.s.c.c.ComputeEngineContainerImpl] Running Community edition
2026.10.20 11:39:49 INFO  ce[][o.s.ce.app.CeServer] Compute Engine is started
2026.10.20 11:39:49 INFO  app[][o.s.a.SchedulerImpl] Process[ce] is up
2026.10.20 11:39:49 INFO  app[][o.s.a.SchedulerImpl] SonarQube is operational
```

Figura 3. *Registro de arranque de SonarQube con el mensaje “SonarQube is operational”.*

Primer ingreso y cambio de credenciales

Ingresamos a `http://localhost:9000` con las credenciales por defecto, y SonarQube nos exigió cambiar la contraseña antes de continuar. Establecimos una contraseña administrativa nueva.

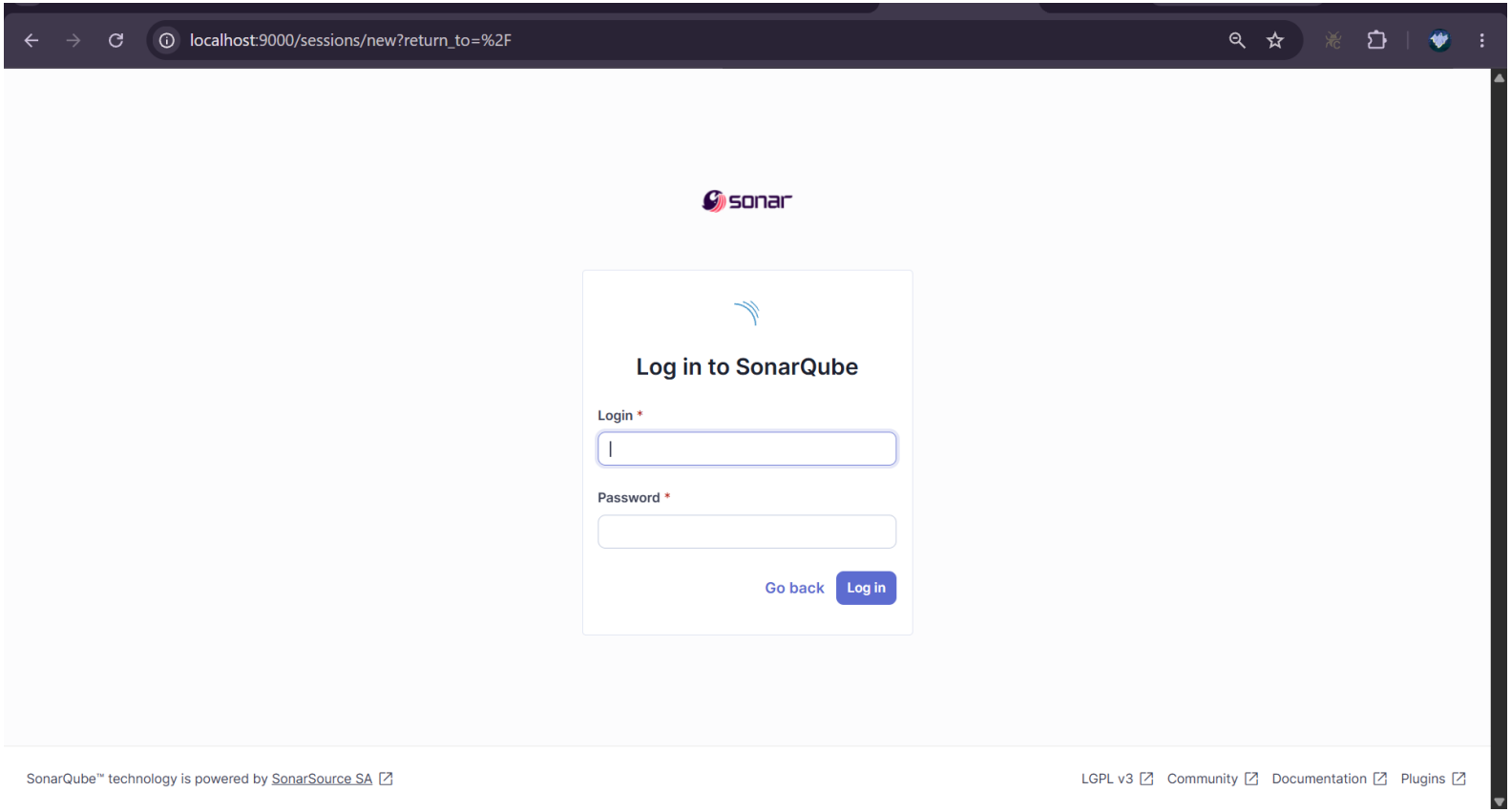


Figura 4. *Pantalla de inicio de sesión de SonarQube en http://localhost:9000.*

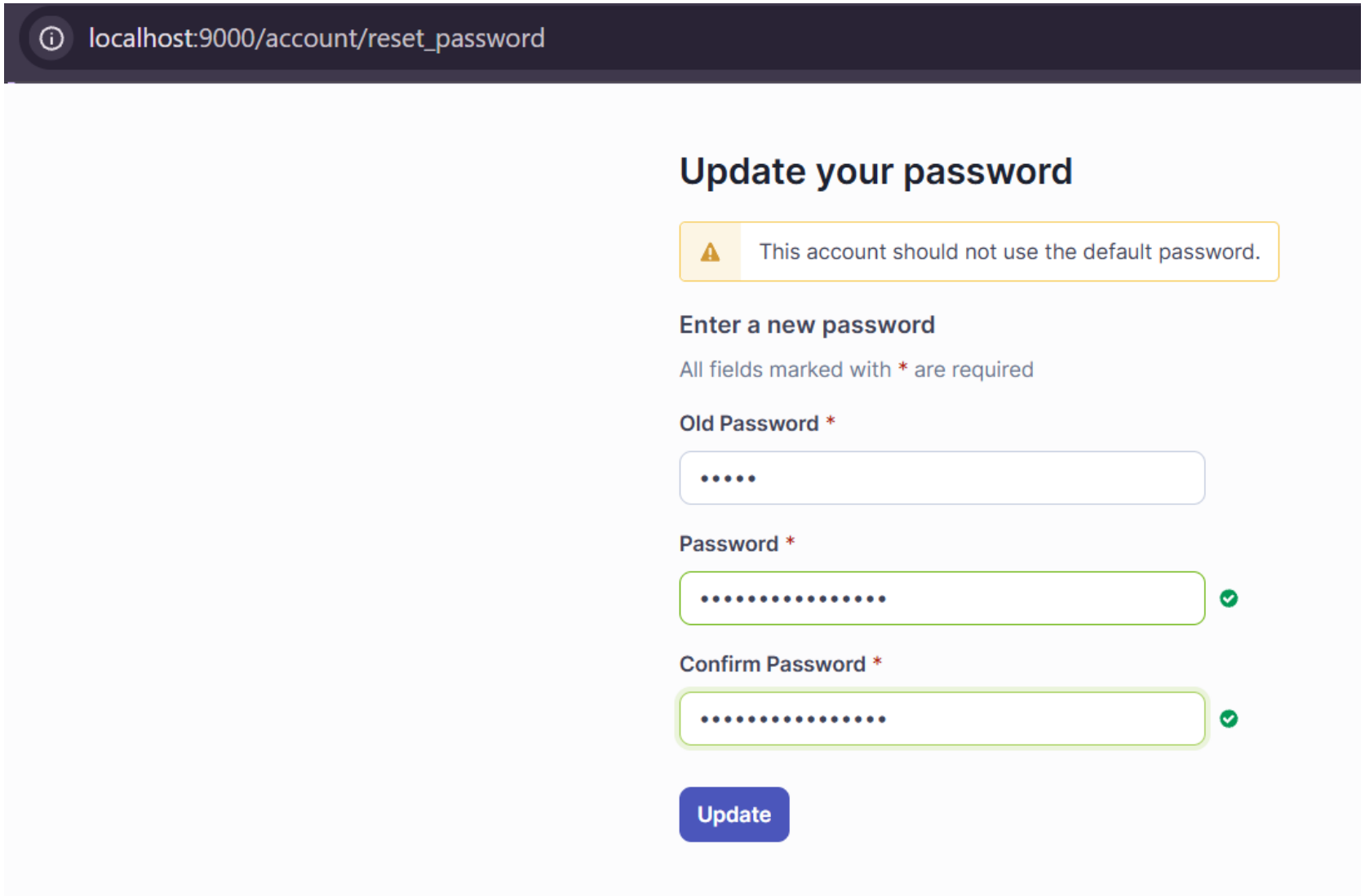


Figura 5. *Pantalla "Update your password": cambio obligatorio de la contraseña por defecto del administrador.*

Creación del proyecto y generación del token

Creamos el proyecto local con nombre y clave `product-api` y rama principal `main`. Después elegimos el análisis local y generamos el token `sonarqube-token`, que Jenkins usa para autenticarse ante SonarQube. Por seguridad, ocultamos el valor del token en la captura.

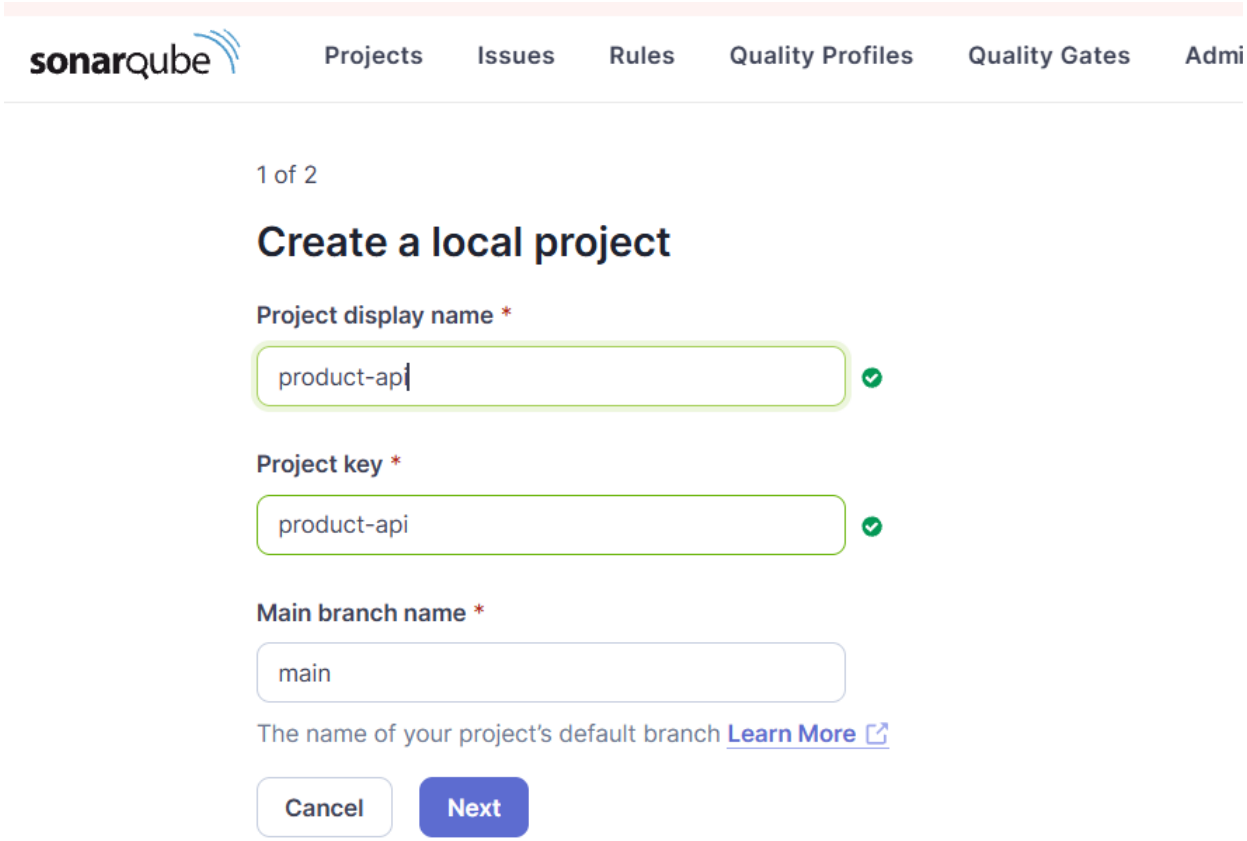


Figura 6. *Creación del proyecto local product-api con project key product-api y rama principal main.*

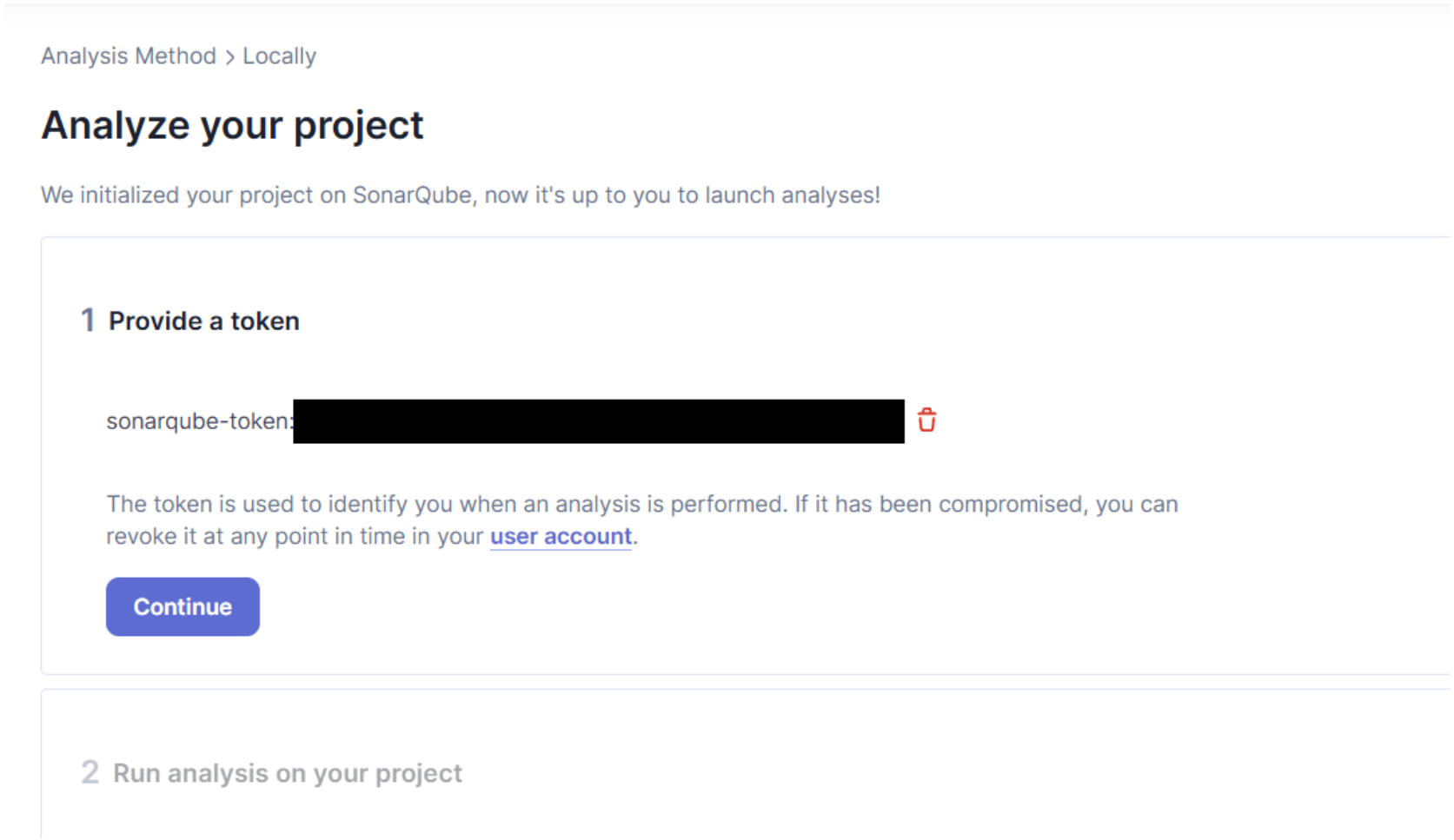


Figura 7. *Generación del token sonarqube-token para el análisis del proyecto (valor ocultado por seguridad).*

Configuración del Quality Gate estricto

Creamos el Quality Gate `Icesi-DevOps-Gate` con condiciones sobre el código nuevo: cobertura mínima de 80 %, ningún bug ni vulnerabilidad, 100 % de los Security Hotspots revisados, duplicación máxima de 3 % y ningún issue. Si alguna condición no se cumple, el análisis se marca como fallido. Luego lo asociamos al proyecto `product-api`.

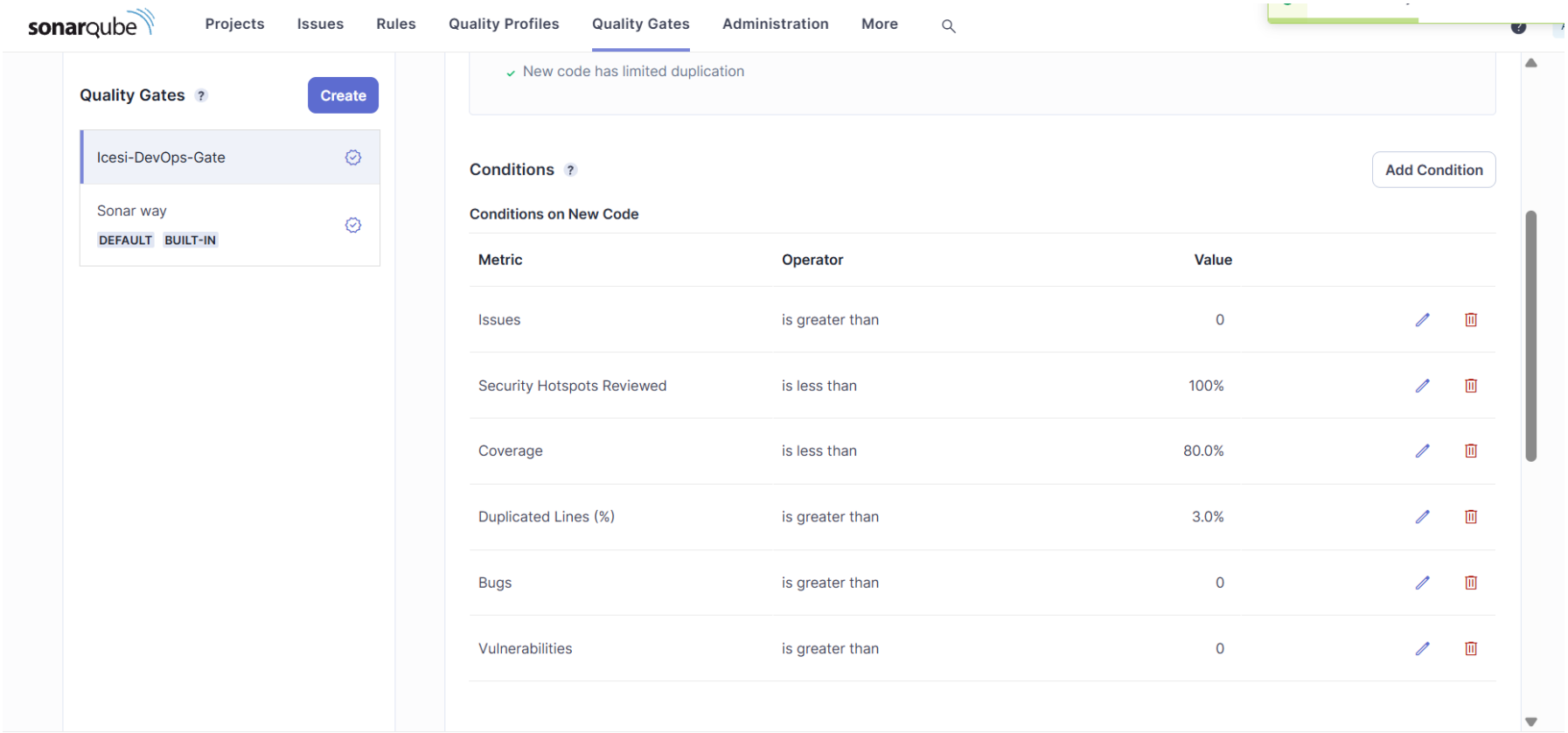


Figura 8. *Condiciones del Quality Gate Icesi-DevOps-Gate sobre el código nuevo (cobertura, bugs, vulnerabilidades, hotspots, duplicación e issues).*

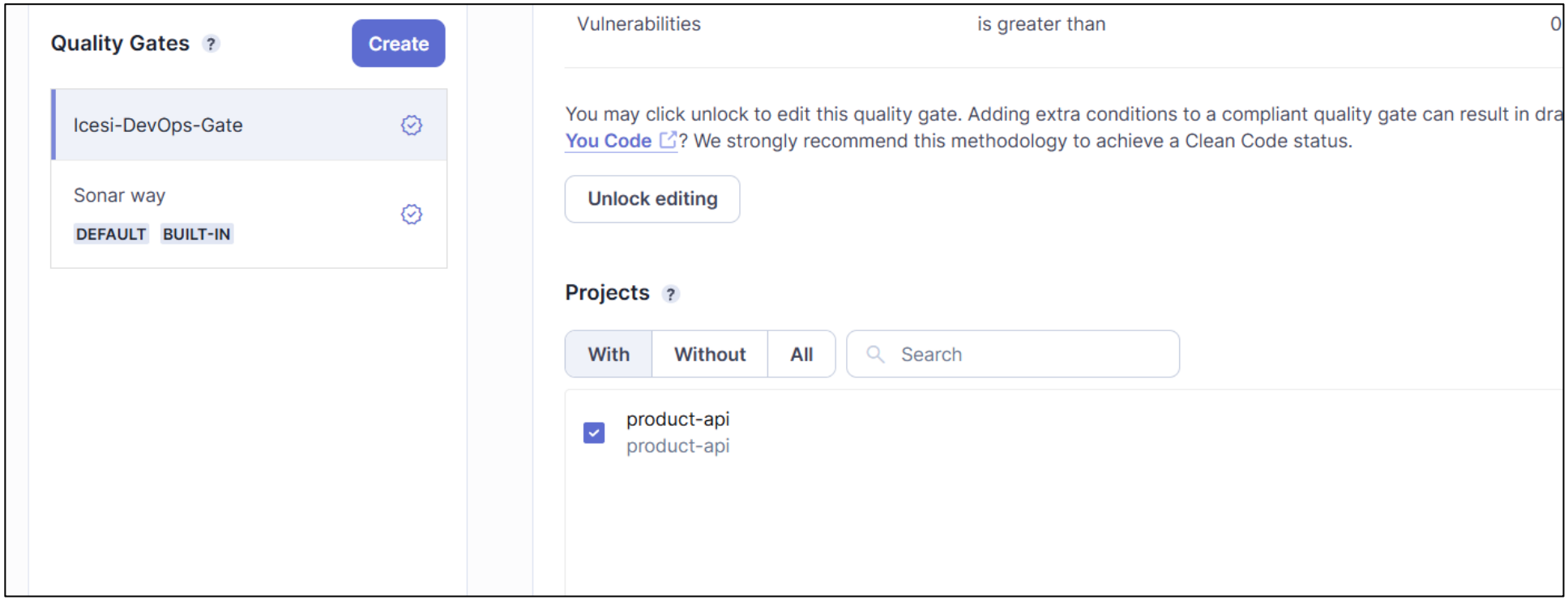


Figura 9. *Asociación del Quality Gate Icesi-DevOps-Gate con el proyecto product-api.*

Webhook hacia Jenkins y credenciales en Jenkins

Para que Jenkins no tenga que consultar repetidamente a SonarQube, configuramos el webhook `Jenkins-Webhook` que apunta a `http://host.docker.internal:9182/sonarqube-webhook/`. Como SonarQube corre dentro de un contenedor, usamos `host.docker.internal` para alcanzar a Jenkins desde él. El webhook tiene secreto configurado y la columna de última entrega aparece como “Never” porque aún no se había ejecutado ningún análisis.

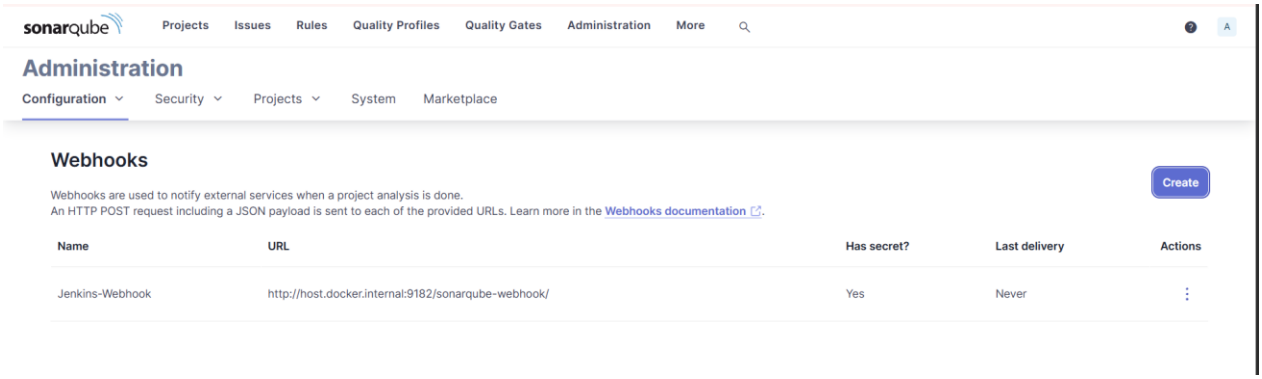


Figura 10. *Webhook Jenkins-Webhook configurado en SonarQube hacia el endpoint /sonarqube-webhook/ de Jenkins.*

En Jenkins registramos las credenciales globales que usa el pipeline: las de Nexus, las de GitHub y el secreto `sonarqube-token` con el token de SonarQube. Con el plugin SonarQube Scanner y el servidor `SonarQube-Server` configurado, el pipeline puede inyectar la URL y el token sin exponerlos en el código.

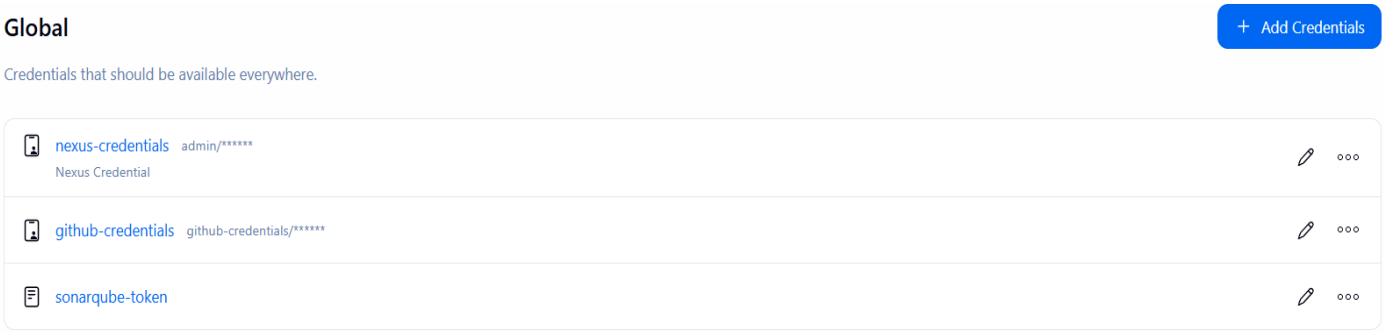


Figura 11. *Credenciales globales en Jenkins: nexus-credentials, github-credentials y sonarqube-token.*

Punto 2.2 – Fase 2: Etapa 1, Test y cobertura con JaCoCo (Hito 3)

La primera etapa del pipeline es el stage “Pruebas & Cobertura JaCoCo”, que ejecutamos dentro del directorio `backend`. Allí corremos `mvn clean verify`, que compila el proyecto, ejecuta las pruebas unitarias y de integración y hace que JaCoCo genere el reporte `target/site/jacoco/jacoco.xml`. El bloque `post { always { junit ... } }` publica los resultados de JUnit en Jenkins incluso si alguna prueba falla, de modo que siempre podamos ver qué aserción falló.

Evidencia

En la vista de la ejecución, el stage terminó en verde en 21 s y mostró tres pasos: el mensaje de la etapa, `mvn clean verify` (20 s) y el archivado de los resultados JUnit. La gráfica de tendencia muestra las 6 pruebas del proyecto aprobadas de forma consistente en las ejecuciones registradas.

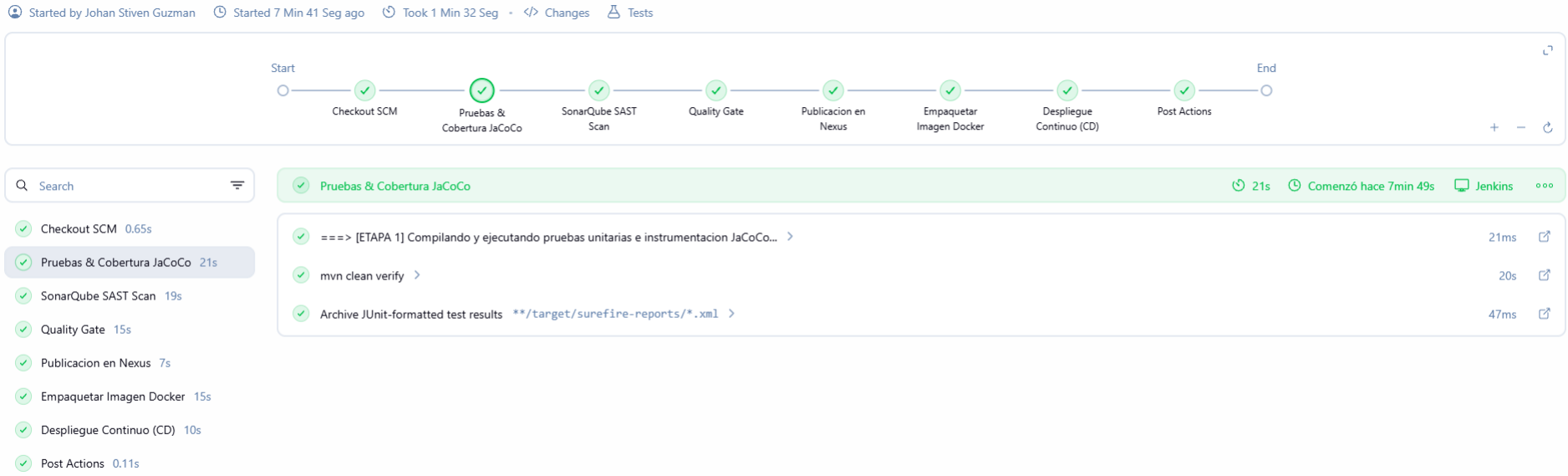


Figura 12. *Stage “Pruebas & Cobertura JaCoCo” en Jenkins: mvn clean verify y archivado de resultados JUnit, con todo el pipeline en verde.*

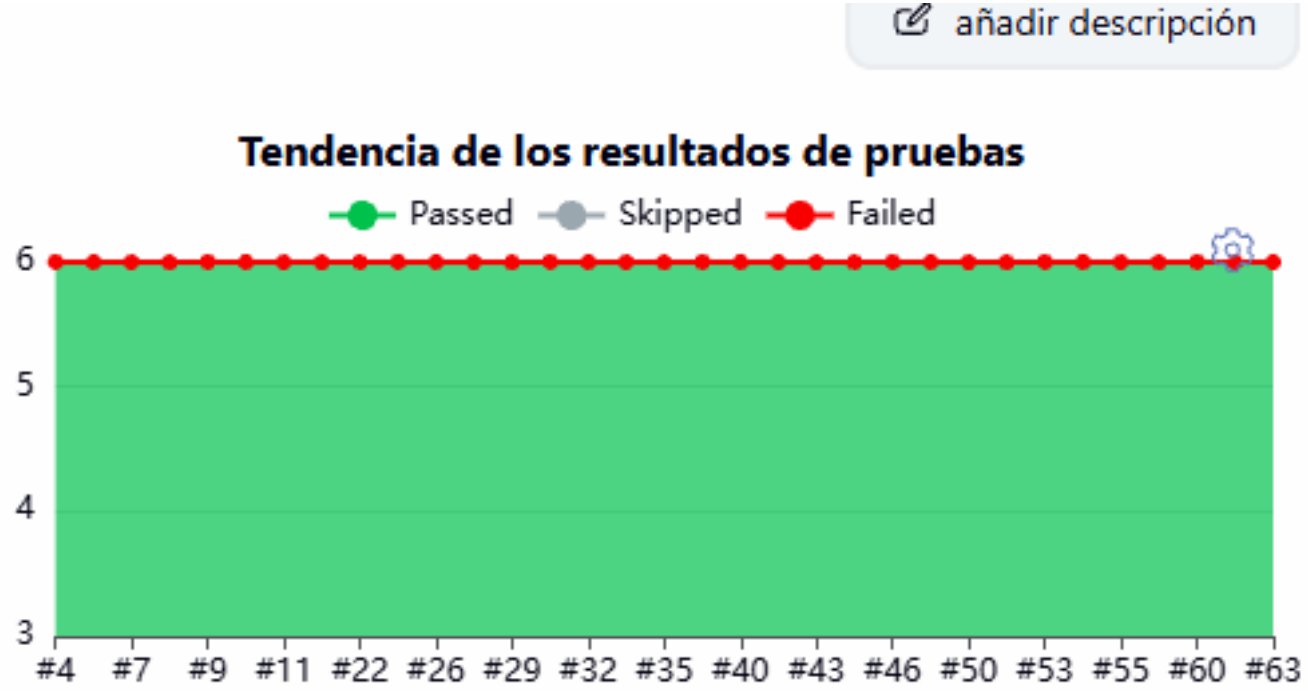


Figura 13. *Gráfica "Tendencia de los resultados de pruebas" en Jenkins con las pruebas aprobadas por ejecución.*

Punto 2.3 – Fase 3: Etapa 2, Validación SAST y Quality Gate (Hitos 4 y 5)

Análisis estático con SonarQube

En el stage “SonarQube SAST Scan” usamos `withSonarQubeEnv("SonarQube-Server")`, que inyecta la URL y el token sin exponerlos en el script. Dentro ejecutamos `mvn sonar:sonar` con el `projectKey` de `product-api` y la ruta del XML de JaCoCo, para que SonarQube también muestre la cobertura. Así enviamos a SonarQube el código, los bugs, los hotspots y la cobertura. El stage terminó en verde en 19 s.



Figura 14. *Stage “SonarQube SAST Scan”: ejecución de mvn sonar:sonar con la clave del proyecto y la ruta del reporte de JaCoCo.*

Quality Gate como compuerta bloqueante

El stage “Quality Gate” envuelve `waitForQualityGate()` en un `timeout` de 5 minutos. El pipeline se pausa hasta que el webhook de SonarQube avisa el resultado; si `qg.status != 'OK'`, se ejecuta `error(...)` y el build se aborta en rojo, y si es `OK`, continúa. Para llegar a verde corregimos los dos defectos deliberados del proyecto: reemplazamos `java.util.Random` por `SecureRandom` (CWE-330) y sustituimos `Optional.get()` por `.orElseThrow(...)` (CWE-476).

Evidencia

La captura muestra el stage “Quality Gate” aprobado: el pipeline esperó el análisis (14 s) y registró el mensaje de aprobación del Quality Gate con estado OK.

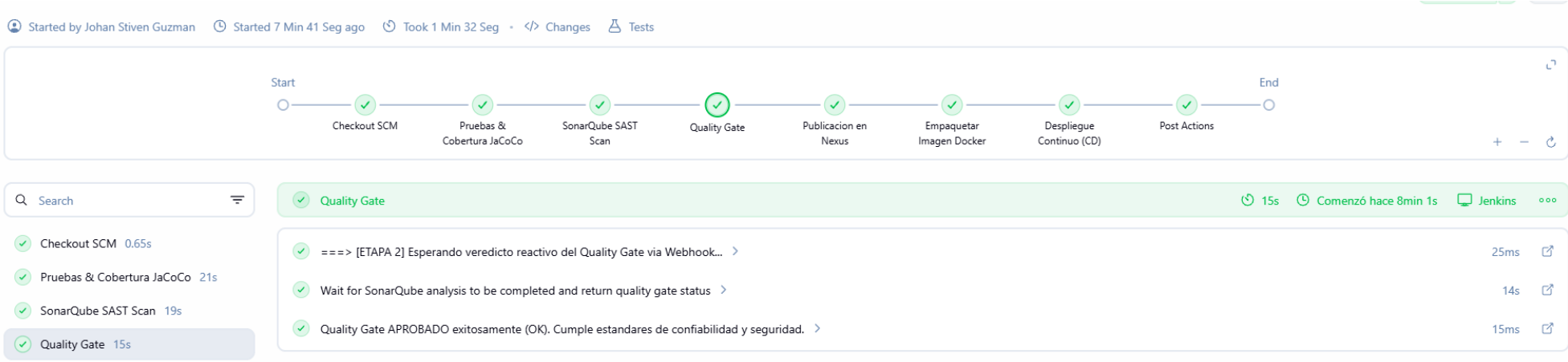


Figura 15. *Stage “Quality Gate” aprobado: espera del análisis de SonarQube y mensaje de Quality Gate APROBADO (OK).*

Punto 2.4 – Fase 4: Etapa 3, Publicación en Nexus e imagen Docker (Hito 6)

Nexus. Con `withCredentials` exponemos las credenciales de Nexus como las variables `NEXUS_USER` y `NEXUS_PASS` y ejecutamos `mvn -B -s settings.xml deploy -DskipTests`. Con `-DskipTests` publicamos el JAR sin repetir las pruebas que ya pasaron en la etapa 1.

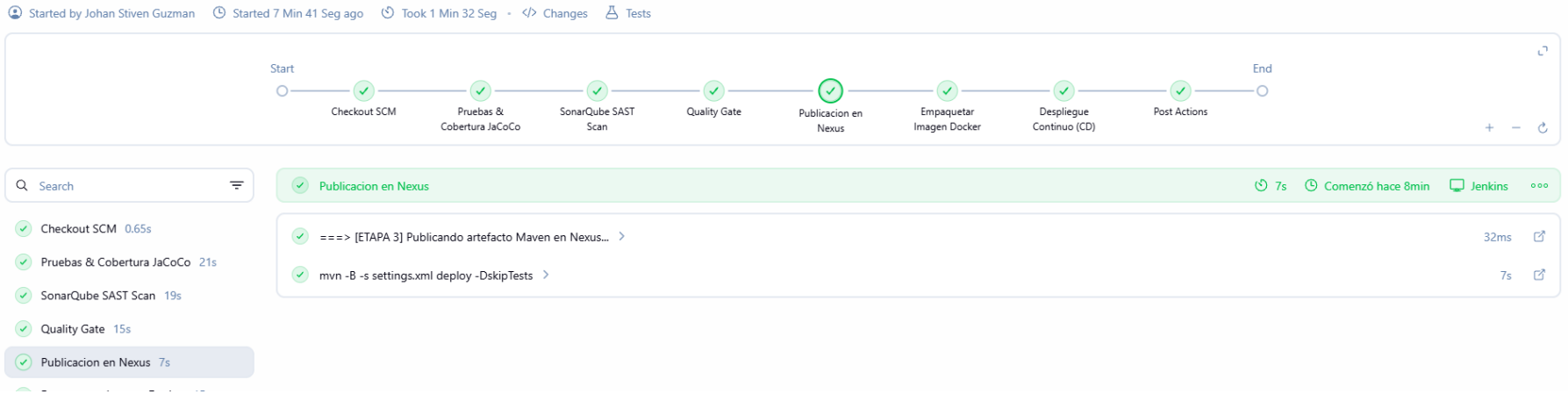


Figura 16. *Stage "Publicación en Nexus": mvn -B -s settings.xml deploy -DskipTests completado en 7 s.*

Docker. Con `docker build -t product-api:${BUILD_NUMBER} -t product-api:latest backend/` construimos la imagen con una etiqueta única por build, que da trazabilidad. En la captura, el build #63 generó la imagen `product-api:63`.

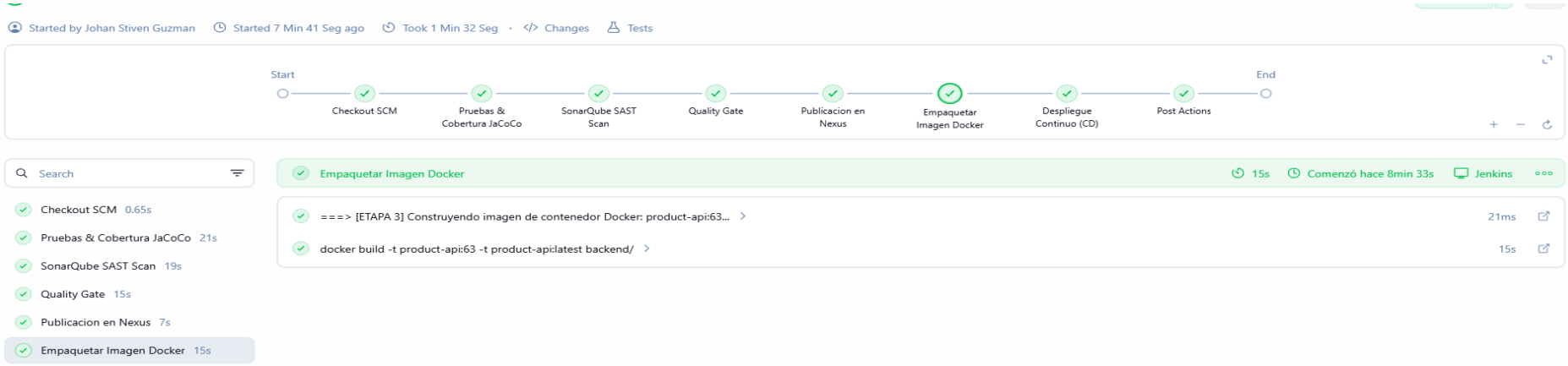


Figura 17. *Stage "Empaquetar Imagen Docker": construcción de la imagen product-api con etiqueta de build (product-api:63) y latest.*

Punto 2.5 – Fase 5: Etapa 4, Despliegue continuo y Healthcheck (Hito 7)

Despliegue idempotente. Para poder ejecutar el pipeline muchas veces sin que falle, primero detenemos y eliminamos la versión anterior del contenedor y después levantamos la nueva:

```
docker stop product-api-running || true
docker rm product-api-running || true
docker run -d --name product-api-running --network cicl_network -p 8085:8080 product-api:${BUILD_NUMBER}
```

Smoke test. Después de desplegar no damos el release por bueno solo porque `docker run` haya terminado bien. Hacemos hasta 10 intentos con `curl -s -f` a `http://product-api-running:8080/api/products`, con 3 s de espera entre cada uno. Usamos el nombre del contenedor porque Jenkins y la API comparten la red Docker `cicl_network`. Además, si el despliegue falla, el pipeline muestra automáticamente un diagnóstico (estado del contenedor, logs de Spring Boot, `docker inspect` y un `curl -v`), una mejora sobre la guía original.

Evidencia

El stage “Despliegue Continuo (CD)” terminó en verde en 10 s, con el despliegue del contenedor y el smoke test (9 s). Como el script solo termina con éxito cuando el endpoint responde HTTP 200, el verde confirma que la API quedó operativa en el puerto 8085.

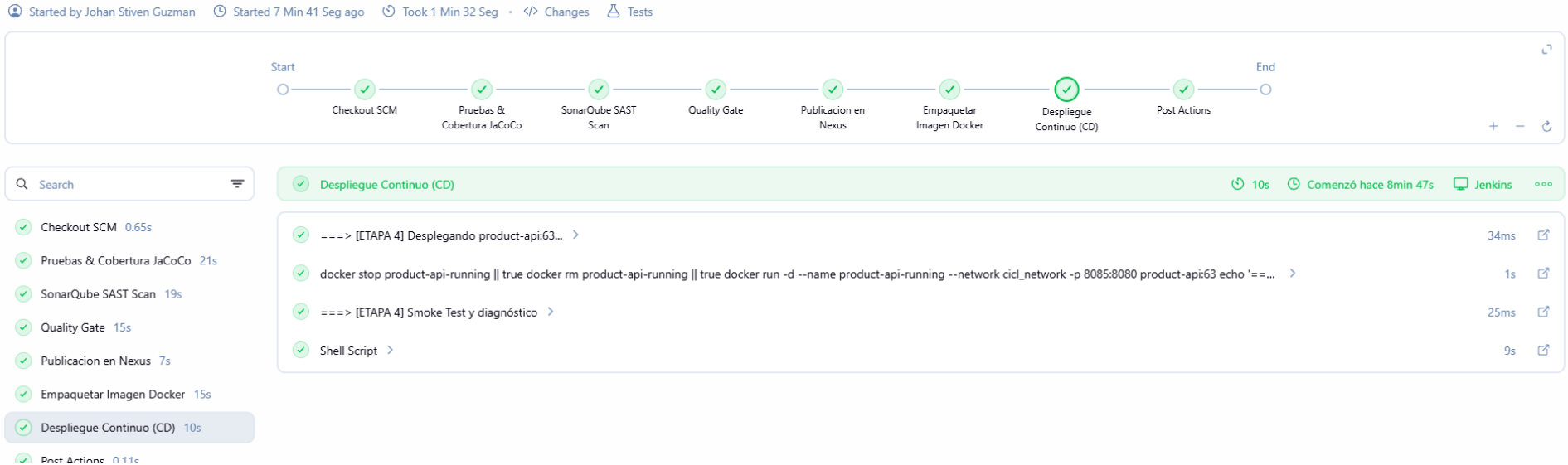


Figura 18. *Stage “Despliegue Continuo (CD)”: despliegue de product-api:63 en la red cicl_network con puerto 8085 y smoke test exitoso.*

Punto 2.6 – Pipeline completo

Finalmente verificamos el pipeline de extremo a extremo. El build #63 pasó las ocho etapas (Checkout SCM, Pruebas & Cobertura JaCoCo, SonarQube SAST Scan, Quality Gate, Publicación en Nexus, Empaquetar Imagen Docker, Despliegue Continuo y Post Actions) en 1 min 32 s, y el bloque `post` mostró el mensaje de pipeline exitoso. En la vista por etapas también se alcanza a ver una ejecución anterior en rojo, parte del proceso de corrección hasta llegar a verde.

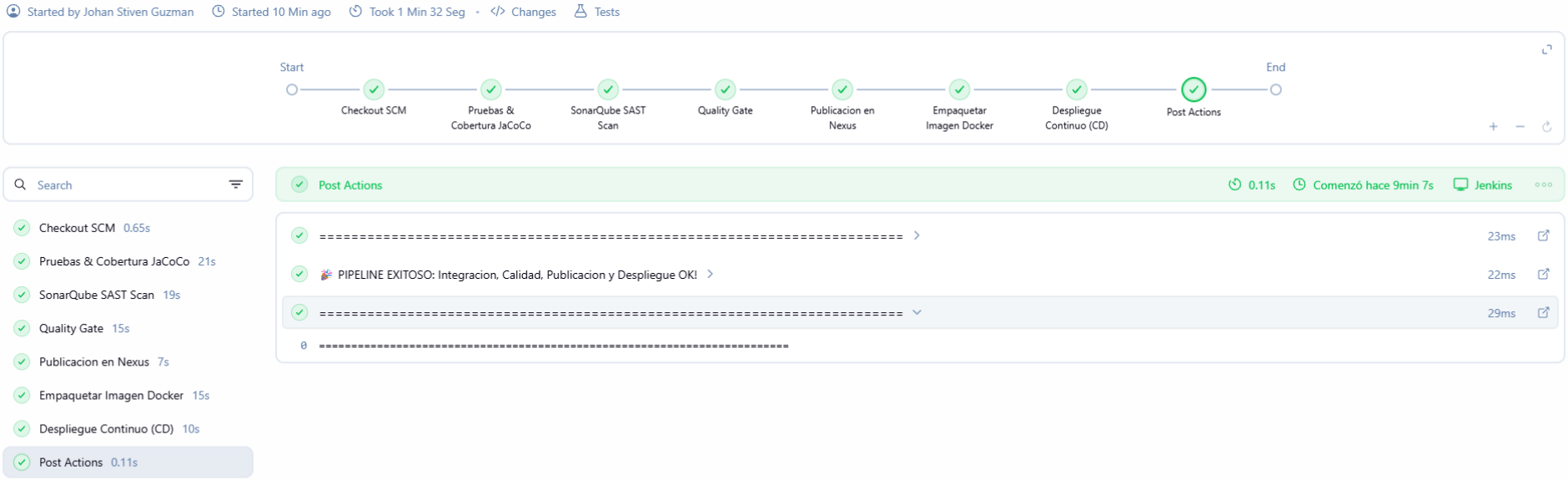


Figura 19. *Pipeline completo en verde en Jenkins: las ocho etapas aprobadas y el mensaje final de Post Actions.*

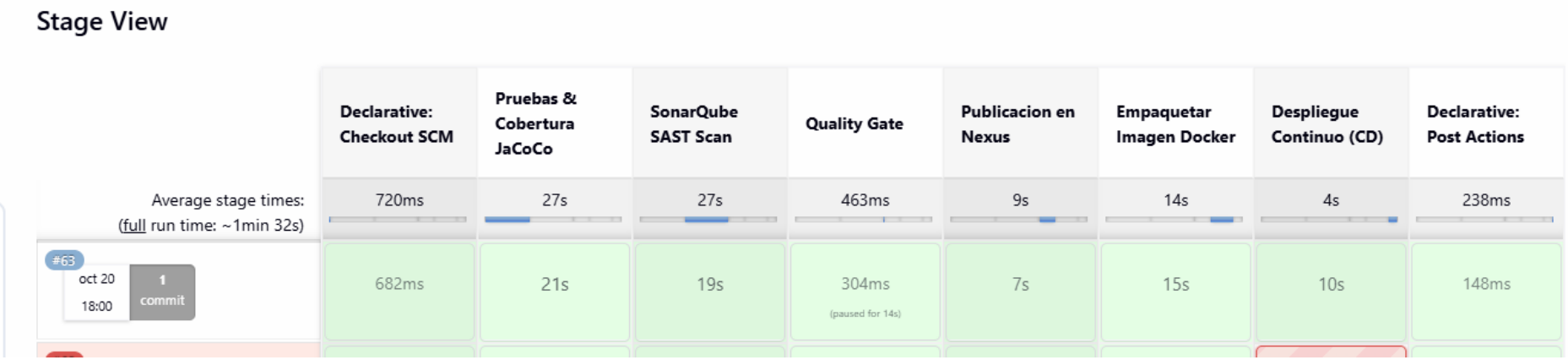


Figura 20. *Stage View de Jenkins con los tiempos por etapa del build #63, todas en verde.*

Punto 3 – Cuestionario de análisis técnico y reflexión DevOps

1. Gobernanza y compuertas de bloqueo

En nuestro pipeline ubicamos el Quality Gate antes de la publicación en Nexus, de la construcción de la imagen Docker y del despliegue. Lo hicimos así porque publicar o desplegar antes de evaluar el Quality Gate invalida el propósito del pipeline: estaríamos distribuyendo código que nadie ha auditado.

- **Impacto técnico:** Nexus es un repositorio inmutable. Un binario con vulnerabilidades como las que trae `product-api` (CWE-330 por usar `java.util.Random` y CWE-476 por llamar a `Optional.get()` sin validar) quedaría publicado, versionado y disponible para que otros equipos lo consuman. Si además se despliega, la vulnerabilidad queda expuesta en ejecución.
- **Impacto de negocio:** riesgo de brechas de seguridad, fuga de datos, multas por incumplimiento normativo y pérdida de reputación. Corregir en producción cuesta mucho más que detectar en CI, que es la idea del *shift-left*.
- Retirar un artefacto que otros ya consumieron es difícil y puede propagar el defecto a otros sistemas.

Para nosotros, tener que corregir los dos defectos deliberados antes de que el pipeline pudiera avanzar fue la demostración práctica de por qué la compuerta debe bloquear.

2. Webhook frente a polling activo

En nuestro pipeline el mecanismo funciona en cuatro pasos:

1. `mvn sonar:sonar` analiza el proyecto y escribe `target/sonar/report-task.txt`, que contiene el `ceTaskId`, el identificador de la tarea en el Compute Engine de SonarQube.
2. `waitForQualityGate()` lee ese archivo y suspende el pipeline sin consumir un ejecutor activo.
3. Cuando SonarQube termina de procesar, envía un POST al endpoint `/sonarqube-webhook/` de Jenkins con el `ceTaskId` y el estado del Quality Gate. En nuestro caso, el webhook `Jenkins-Webhook` apunta a `host.docker.internal:9182`.
4. Jenkins asocia ese `ceTaskId` con el build que está esperando, lo despierta y `waitForQualityGate()` devuelve el resultado.

Esta arquitectura es superior al polling porque:

- evita peticiones repetitivas que casi siempre responden “aún no”;
- reduce la carga en Jenkins y en SonarQube cuando hay muchos builds concurrentes;
- la respuesta es inmediata, sin esperar al siguiente ciclo de consulta;
- escala mejor en un clúster de CI/CD.

3. Resiliencia con timeout

El webhook depende de que la red y SonarQube funcionen. Si hay un fallo de red, SonarQube se reinicia o el webhook nunca llega, Jenkins esperaría indefinidamente una notificación que no vendrá. Eso genera *Zombie Builds*: builds colgados que:

- ocupan ejecutores y recursos;
- bloquean la cola y otros despliegues;
- dejan al equipo sin retroalimentación.

Por eso, en nuestro `Jenkinsfile` envolvimos `waitForQualityGate()` en `timeout(time: 5, unit: 'MINUTES')`. Actúa como *circuit breaker*: tras 5 minutos aborta el stage, falla rápido, libera recursos y nos obliga a investigar. En nuestras ejecuciones el stage tardó cerca de 15 s, así que el límite es holgado y solo se activaría ante un problema real. Aplicamos el principio *fail fast*: ningún paso que dependa de un sistema externo debe poder bloquearse para siempre.

4. Falso positivo en despliegues y rol del Healthcheck

`docker run -d` retorna 0 en cuanto el contenedor se crea y el proceso inicia, no cuando la aplicación está funcionando. Spring Boot tarda varios segundos en arrancar y muchos fallos solo aparecen después:

- error al construir el contexto de Spring (por ejemplo, un bean mal configurado);
- fallo de conexión a la base de datos;
- puerto ocupado;
- falta de variables de entorno o de memoria (OOM);
- el contenedor muere segundos después con `ExitCode != 0`.

Sin una verificación adicional, nuestro pipeline podría declarar éxito con la aplicación caída. Por eso el smoke test consulta activamente `/api/products` con `curl -f` y solo da por bueno el despliegue si responde HTTP 200. Si no responde tras los 10 intentos, el build falla y muestra el diagnóstico (logs e `inspect`), lo que protege la estabilidad del entorno.